

Tom's Survival Tips

Just a few colloquialisms I've accumulated through the years.

- How you get the answer is more important than the answer you get.
- If it doesn't work for me, it doesn't work.
- Just because it works doesn't mean it's right.
- There are no "good" warnings
- It doesn't matter how pretty it is, if it's broken nobody will buy it.
- Never compile a header file, and never #include a source file
- Never occupy memory in a header file
- Nothing lives in the global, unnamed namespace unless you put it there – don't! Abstain from "using namespace std;"
- Don't mix deliverable files with working files – keep your deliverables (output text, source and header files) in a separate directory away from build, executables, make, solution, git, object, and all other non-deliverable files.
- Don't confuse Visual Studio Code with Visual Studio – they are very different tools
- We C++ programmers are a weird breed. We like to ...
 - point at things that aren't there (dangling pointers)
 - talk to ghosts (dereference null pointers)
 - throw big temper tantrums and hurl sharp daggers when we get confused and don't get our own way (exceptions)
 - hide and protect our private parts (access control)
 - strive to have no friends, but the friends we have get benefits (friendships)
 - hoard stuff we can't use (memory leaks)
 - be one off. We start counting things from zero instead of one (off-by-1 error)
 - love compiler error messages (discover and fix errors early)

Setting up your C++ Development Environment

1. On a Chromebook?

Sorry, no one that I'm aware of has been able to use a Chromebook either for software development or remote desktops. You really need a different machine.

2. On a Mac?

The Mac is a great choice for those that have a Mac and know how to use it. I do not. I offer no advice or support. You're on your own. Sorry.

- Consider borrowing a PC from CSUF, installing Tuffix as a VM, or (as a last resort) using the [Virtual Computing Lab \(VCL\)](#).
 - Select the "ECS Ubuntu 2004 4x10 Tuffix 2023v1" environment option.
 - [Virtual Computing Lab Guide](#)
 - [VCL Tuffix Demo](#)
 - Note: All data, configurations, and work performed will be lost when your reservation expires. Be sure to save your work to remote storage (local to your host machine, Google Drive, GitHub, etc)
 - Note: As of today (3-AUG-2023) the VCL machines are still on Ubuntu 20.04. You must upgrade to 22.04 each time you request a new reservation. See Tuffix Support below.

3. Don't have a PC?

- Borrow one from CSUF
 - Performing a native install? Request a blank loaner laptop.
- [Long-Term Laptop Checkout](#) and other [Equipment Checkout](#) are available.

4. On a PC?

Step 1 of 3 - Select a platform

First, verify virtualization is enabled in your BIOS/UEFI

- [How to check virtualization enabled on Windows 10 or 11 without accessing BIOS menu \(how2shout.com\)](#)
 - Note: Many Windows 10 PCs—and all PCs that come preinstalled with Windows 11—already have virtualization enabled.
 - If “Virtualization” is set to “Enabled”, you're good to go - Your BIOS/UEFI is configured properly.
- [Enable virtualization on Windows 11 PCs - Microsoft Support](#)
- [How To Enable Virtualization In BIOS \(technewstoday.com\)](#)

If you are running Windows as a guest VM over Hyper-V, enable nested virtualization manually from the host machine. Shutdown the VM, run PowerShell as administrator, and then at the command line type:

- `Set-VMProcessor -VMName <VMName> -ExposeVirtualizationExtensions $true`
 - See [Troubleshooting Windows Subsystem for Linux | Microsoft Learn](#)

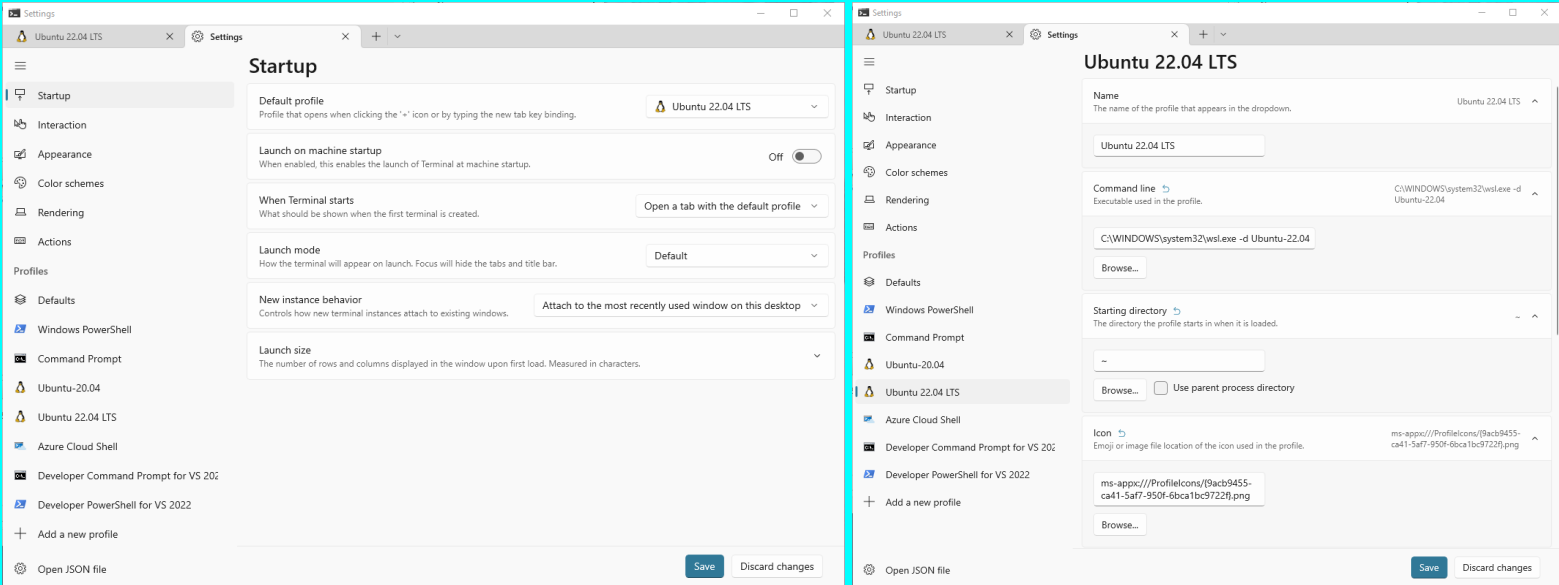
Option 1 (Recommended): Use Windows Subsystem for Linux (WSL2)

- Windows 10: Upgrade to Windows Terminal (Windows 11 already uses Windows Terminal)
 - Get and install Windows Terminal (caution: not Windows Terminal Preview) from the Microsoft Store. Open the store and search for “Terminal”, select Windows Terminal, click “Get”.
 - [Windows Terminal installation | Microsoft Docs](#)
 - [Windows Terminal Startup Settings | Microsoft Learn](#)
 - [An overview on Windows Terminal | Microsoft Docs](#)
- Install WSL components from an elevated Windows PowerShell command line
 - `wsl --install --distribution Ubuntu-22.04`
 - Reboot Windows. Login. WSL installation will resume and may take a few minutes. Eventually ...
 - Enter your desired UNIX username and password when prompted (unrelated to your Windows username and password)

If you get an error message, see “verify virtualization is enabled” at the beginning of this section and also [Troubleshooting Windows Subsystem for Linux](#)

- At the Linux command prompt, type the following and note your distribution’s name
 - `wsl.exe --list --verbose`
 - Should see something like:

NAME	STATE	VERSION
* Ubuntu-22.04	Running	2
- Close the window by typing: `exit`
- Configure your Terminal Window
 - [Windows Terminal Customization for WSL2 - The Complete Guide \(ceos3c.com\)](#)
 - Open a Terminal Window, and navigate to the Setting tab. I tweak the default settings slightly:
 - Startup | Default profile = Ubuntu 22.04 LTS (I also had something called Ubuntu-22.04, which I ignored)
 - Ubuntu 22.04 LTS | Name = Ubuntu 22.04 LTS
 - Ubuntu 22.04 LTS | Command line = `C:\WINDOWS\system32\wsl.exe -d Ubuntu-22.04`
 - Ubuntu-22.04 is the name of my distribution from `wsl.exe --list --verbose`
 - Enables you to open a Terminal window at your current location (path). Right-click in a File Explorer window and select “Open in Terminal” or shift-right-click and select “Open Linux shell here”
 - Ubuntu 22.04 LTS | Starting directory = `~`
 - If you want the default to be your Windows home directory instead of your Linux home directory, I’ve also seen `= %USERPROFILE%`
 - Ubuntu 22.04 LTS | Icon = `ms-appx:///ProfileIcons/{9acb9455-ca41-5af7-950f-6bca1bc9722f}.png`



- Additional resources
 - [Set up a WSL development environment | Microsoft Learn](#)
 - [Ubuntu 22.04 on WSL \(Windows Subsystem for Linux\) - Linux Tutorials - Learn Linux Configuration](#)
 - [Troubleshooting Windows Subsystem for Linux | Microsoft Learn](#)
 - [Windows Subsystem for Linux Documentation | Microsoft Learn](#)
 - [Install WSL on Windows 10 | Microsoft Docs](#)
 - [Using WSL 2 with Visual Studio Code](#)
 - [VSCode in WSL2-Ubuntu. How to setup VSCode in Ubuntu 20.04 LTS... | by Florian Kromer | Medium](#)
 - [How to setup Windows Subsystem Linux \(WSL 2\) with Visual Studio Code on Windows 10 | by Hung Om | Nerd For Tech | Medium](#)

Option 2: Perform a native installation

- Install [Ubuntu Desktop 22.04.1 LTS](#) on a second laptop by following the 12 steps from here: [Install Ubuntu desktop | Ubuntu](#).
 - Note: All data on the computer will be **deleted forever** including all your programs like Microsoft Word and Excel. You cannot reinstall these programs. Please watch the [Tuffix installation videos](#) and ask questions on the [CSUF Tuffix channel](#).
 - Tip: Setup WiFi - if you're using Eduroam, use the [Tuffix Eduroam Authentication Instructions](#) or <https://github.com/mshafae/tusk/blob/main/docs/eduroam.md>
- If you don’t want to carry around a second laptop and you have a spare internal harddrive (recommended, even if you need to buy a small one) or lots of extra disk space in your existing laptop, consider dual booting. This is convenient, but setup is not for the squeamish. Use caution. I recommend removing your Windows harddrive, installing Ubuntu to your second drive, and then replacing your Windows drive.
 - Make sure you have a good full image backup before attempting. Better safe than sorry if something goes wrong.
 - [How to Dual Boot Windows 11 and Ubuntu \(windowsreport.com\)](#)
 - [How to Dual Boot Windows 10 and Ubuntu – Linux Dual Booting Tutorial \(freecodecamp.org\)](#)
 - [OS Prober is Disabled in Ubuntu 22.04, Here's a Workaround - OMG! Ubuntu! \(omgubuntu.co.uk\)](#)
 - [Changing default boot option in grub2 - Ask Ubuntu](#)
 - [How to Set Grub to Remember Your Last Selected Entry - Make Tech Easier](#)
 - [How to Fix Time Differences in Ubuntu 16.04 & Windows 10 Dual Boot | UbuntuHandbook](#)

Option 3: Use a Virtual Machine

- Requires a lot of memory and a beefy machine, but is very convenient as it allows both Windows and Ubuntu to run at the same time and share files.
- I really really dislike Virtual Box, avoid it if possible. Use Hyper-V instead.
- Quick create (recommended): [Windows 11 Hyper-V: Ubuntu 22.04 LTS Installation Procedure](#)
- [How to Install Ubuntu 22.04 Linux on Windows with Hyper-V - Linux Shout \(how2shout.com\)](#)

Option 4: Follow the Tuffix 2022 installation instructions

- CSUF's Computer Science Department's official GNU/Linux development environment is Tuffix. Tuffix is Tuffy the Titan's Linux distribution.
- Instructions on how to install Tuffix either natively or in a VM using VirtualBox are online at <http://csufcs.com/tuffixinstall>. The Tuffix home page is <https://github.com/kevinwortman/tuffix>.

Step 2 of 3 - Update OS and install compilers

In all cases, once you have Ubuntu installed

- Verify you are using Ubuntu 22.04.* with `lsb_release -a`
- Update to the latest security updates, etc.. From the Ubuntu terminal window command line, execute all these in order (see [How to install updates via command line? - Ask Ubuntu](#)):
 - `sudo apt update` # Fetches the list of available updates
 - `sudo apt upgrade` # Installs some updates; does not remove packages
 - `sudo apt full-upgrade` # Installs updates; may also remove some packages, if needed
 - `sudo apt autoremove` # Removes any old packages that are no longer needed
 - `sudo apt clean` # Removes cached packages freeing up disk space
- Run Build.sh to install our compilers. See [C++ Development Root Collection](#) below.

Step 3 of 3 - (Recommended) Install and configure VSCode. See below.

Tuffix Support

Students enrolled in CSPC 120, CPSC 121, and CPSC 131 are recommended to use the Computer Science Department's official GNU/Linux development environment, Tuffix. Tuffix is Tuffy the Titan's Linux distribution. Instructions on how to install Tuffix natively or a Tuffix based VM are online at [Tuffix 2020 Installation Instructions](#). The Tuffix home page is <https://github.com/kevinwortman/tuffix>.

Note that Tuffix 2022 (released January 2023) is based on Ubuntu 22.04. Those using Ubuntu 20.04, either from prior semesters or a fresh install, must upgrade to Ubuntu 22.04. Shy of a dedicated machine hosting a native Tuffix install, my recommended platform is Ubuntu 22.04 hosted on Windows using WSL.

Those using Ubuntu 20.04, either from prior semesters or a fresh install, must upgrade to Ubuntu 22.04 or better. To move to Ubuntu 20.04 to 22.04 without reinstalling from scratch you can:

- Upgrade your existing system in place. See <https://youtu.be/2Mwo4BfJuvA>. Type at the command prompt:
 - `sudo do-release-upgrade`Hint: Ctrl-i, the arrow keys, and the Enter key navigates through those blue full-screen menus

- CSUF TUFFIX Slack Workspace

Students using Tuffix should join the CSUF TUFFIX slack workspace at <https://csuf-tuffix.slack.com>. Please use the #general channel to ask about troubleshooting installing and using Tuffix.

I/O Redirection

- Redirection is: [Wikipedia definition](#)
- [An Introduction to Linux I/O Redirection](#)
- [How to use command redirection under Linux](#) (works on Windows too)
- Examples:
 - `myProject < input.txt > output.txt` # std::cin takes input from input.txt and std::cout sends output to output.txt
 - `myProject < input.txt | tee output.txt` # std::cin takes input from input.txt and std::cout sends output to another program through a pipe
- [Redirection in Linux - Linux Tutorial 8](#)
- [Linux Command Line Tutorial For Beginners 5 - I/O Redirection](#)

How to Build and Run Your Programs with Build.sh

- Compiling, linking, and executing your projects prior to delivery is critical to your success. It's imperative the compiler options you use match what will be used to grade your homework, and there are many - too many to type by hand over and over. Build.sh captures those options so you don't have to worry about getting it right each and every time. There is nothing magical about Build.sh - you could type the command yourself - it just saves you time while reducing typing errors.
 - Build.sh may attempt to validate and then update your compiler versions. You should allow this one-time update to occur.
 - Hint: Ctrl-i, the arrow keys, and the Enter key navigates through those blue full-screen menus
- Latest version is part of my [C++ Development Root](#) collection.
- [How to Build and Run Your Programs with Build.sh](#)

C++ Development Root Collection

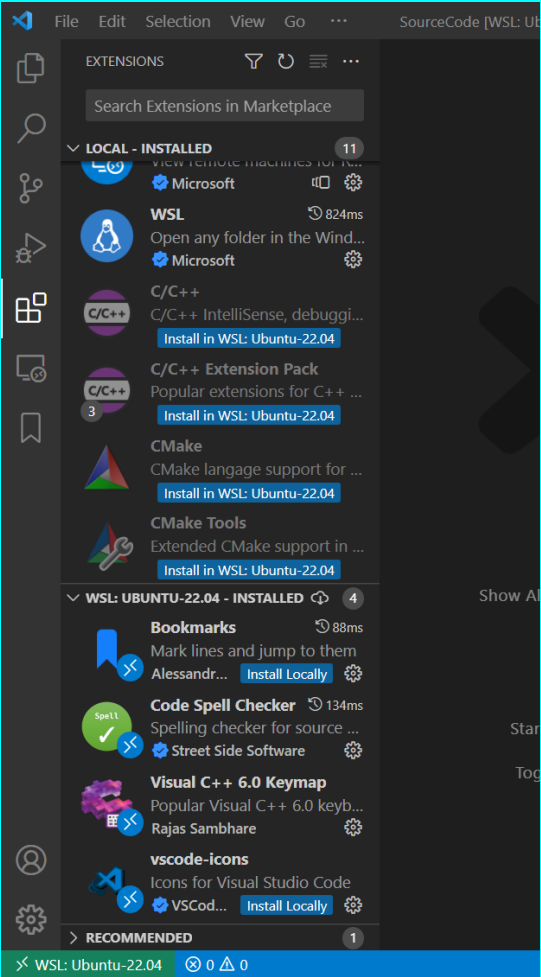
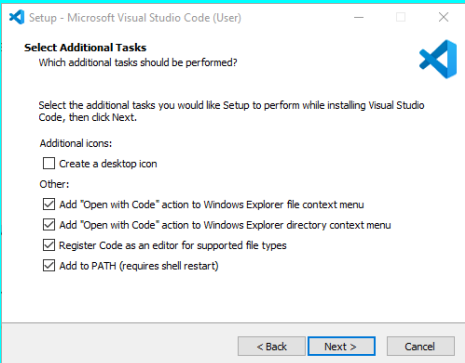
My [collection of folders and files](#) containing development environments for Windows and Linux:

- *MSVC++v17 (2022) Solution Space* - Contains a preconfigured Visual Studio Solution and Project space. Do not move or modify anything under this directory if you're using Visual Studio on Windows. Delete this directory if you're using Linux.
- *SourceCode* - Set your current working directory here. This is where you create all your source and header files. It is also where your input and output files reside.
 - *.vscode* - Contains a preconfigured for Linux VSCode workspace. Do not move or modify anything under this directory if you're using VSCode on Linux. Follow the directions at the top of *user_settings.json*. Delete this directory if you're using Windows
- *.clang-format and .clang-tidy* - These files contain my settings for formatting C++ code and semantic checking. VSCode (Linux) and Visual Studio (Windows) will find and use these files in this location. You may modify these as desired, but this is what I use.
- ***Build.sh*** - a Bash script (Linux) to compile and link your C++ solutions twice - once with g++ and once with clang++. This script will find every source file in the current directory and below and compile all of them as a single program. It uses the required command line options for this course. The grading process will compile and link your delivery using this script. Clean compiles (no warnings and no errors) are expected. I'm not a Mac user but I've been told Build.sh works on Macs if you actually install g++. By default, g++ on the Mac runs clang++, so I've been told.
- *Visual Studio.Ink* - Click this shortcut on Windows to open Visual Studio using my preconfigured Solution and Project. Delete this file if you're using Linux.

VSCode

Option 1: VSCode on WSL

- Install [VSCode on Windows](#) (yes on Windows, not Linux/WSL. VSCode runs on Windows and remotely connects to Linux/WSL)
 - From the Select Additional Tasks, be sure “Add to PATH (requires shell restart)” is checked
- Restart your shell by logging out, and logging back into Windows
- Then, from [your WSL terminal](#) command line
 - Download and unpack the [C++ Development Root](#) collection if you’ve not done so already
 - `cd <your "C++ Development Root/SourceCode" directory>`
 - `../Build.sh` # makes sure the C++ compilers are installed and up-to-date
 - `code --install-extension ms-vscode.cpptools-extension-pack`
 - `code --install-extension ms-vscode-remote.vscode-remote-extensionpack`
 - `code --install-extension streetsidesoftware.code-spell-checker`
 - `code --install-extension sambhare.vc6-keybindings`
 - `code --install-extension vscode-icons-team.vscode-icons`
 - `code .` # Note the dot at the end. This opens VSCode on Windows. Tip: always open VSCode from WSL, never Windows
 - `cp ".vscode/user_settings.json" "$HOME/.vscode-server/data/Machine/settings.json"`
 - `cp ".vscode/keybindings.json" "$HOME/.vscode-server/data/Machine/keybindings.json"`
 - “user_settings.json” and “keybindings.json” are part of the [C++ Development Root Collection](#) above.
 - Open VSCode’s Extensions panel and verify extensions have been installed. Click all “Install in WSL” and “Install Locally” buttons.



- Visit [Developing in the Windows Subsystem for Linux with Visual Studio Code](#) for detailed information, tips, and help

Option 2: VSCode on Linux (VM or native)

- My favorite development environment on Linux is Visual Studio Code with the gdb. It's very easy to install on Ubuntu, and with my preconfigured environment (uses all the right compiler switches) it should get you up and running productively quickly. Install VSCode and C++ extensions from command line:
 - `cd <your "C++ Development Root/SourceCode" directory>`
 - `../Build.sh` # makes sure the C++ compilers are installed and up-to-date
 - `sudo snap install code --classic`
 - Install all 5 extensions listed above (e.g., `code --install-extension ...`)
 - `code .` # Note the dot at the end
 - `cp ".vscode/user_settings.json" "$HOME/.config/Code/User/settings.json"`
 - `cp ".vscode/keybindings.json" "$HOME/.config/Code/User/keybindings.json"`
 - “user_settings.json” and “keybindings.json” are part of the [C++ Development Root Collection](#) above.
- [Visual Studio Code on Linux, Install Visual Studio Code on Ubuntu 22.04 \[Step-by-Step\]](#)
- [Debugging C/C++ with Visual Studio Code](#)